

线性回归

概念

- 仿射变换：通过加权和对特征进行线性变换，并通过偏置项来进行平移
 - 相当于无隐藏层的神经网络
 - 核心假设：线性

核心公式

对于单个数据而言，其中

- 所有特征放到向量 $\mathbf{x} \in \mathbb{R}^d$ 中，
- 并将所有权重放到向量 $\mathbf{w} \in \mathbb{R}^d$ 中

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b.$$

若对于多个数据而言，则

- 用符号表示的矩阵 $\mathbf{X} \in \mathbb{R}^{n \times d}$ 引用数据集的 n 个样本。
- $\mathbf{X}$ 的每一行是一个样本，每一列是一种特征。

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

损失函数

回归问题常用平方误差函数

当样本 i 的预测值为 $\hat{y}^{(i)}$ ，其相应的真实标签为 $y^{(i)}$ 时，平方误差可以定义为以下公式：

$$l^{(i)}(\mathbf{w}, b) = \frac{1}{2} \left(\hat{y}^{(i)} - y^{(i)} \right)^2.$$

为了度量模型在整个数据集上的质量，我们需计算在训练集 n 个样本上的损失均值

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n l^{(i)}(\mathbf{w}, b)$$

随机梯度下降SGD

- 在每次迭代中，我们首先随机抽样一个**批量大小** $\mathcal{B}$
- 然后，我们计算小批量的平均损失关于模型参数的导数（也可以称为梯度）。

- 最后，我们将梯度乘以**学习率** η ，并从当前参数的值中减掉。

我们用下面的数学公式来表示这一更新过程（ ∂ 表示偏导数）：

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b).$$

总结一下，算法的步骤如下：

- 初始化模型参数的值，如随机初始化；
- 从数据集中随机抽取小批量样本且在负梯度的方向上更新参数，并不断迭代这一步骤。

对于平方损失和仿射变换，我们可以明确地写成如下形式：

$$\begin{aligned} \mathbf{w} &\leftarrow \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{\mathbf{w}} l^{(i)}(\mathbf{w}, b) = \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right), \\ b &\leftarrow b - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_b l^{(i)}(\mathbf{w}, b) = b - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right). \end{aligned}$$

核心代码

- 导入nn，即neuro network

```
from torch import nn
```

- 定义一个模型变量net，是Sequential类的实例
 - 第一个参数为输入特征形状
 - 第二个参数为输出特征形状

```
net = nn.Sequential(nn.Linear(2,1))
```

- 初始化模型参数， w 服从 $N(0, 0.01^2)$ ， b 为0

```
net[0].weight.data.normal_(0,0.01)
net[0].bias.data.fill_(0)
```

- 定义损失函数MSE

```
loss = nn.MSELoss()
```

- 定义优化算法：随机梯度下降SGD

```
trainer = torch.optim.SGD(net.parameters(),lr=0.03)
```

- 训练

```
num_epochs = 3 # 定义训练轮数
for epoch in range(num_epochs):    # 外层循环：训练轮数
    for X, y in data_iter:         # 内层循环，一批一批训练
        l = loss(net(X),y)         # 前向传播，算Loss
        trainer.zero_grad()        # 清空上一步的梯度
        l.backward()               # 反向传播，计算梯度
        trainer.step()             # 更新梯度
    l = loss(net(features), labels) # 一次训练结束后，看看损失
    print(f'epoch{epoch+1}, loss{l:f}')
```

Softmax回归

以预测2x2图片里的是"猫"、"鸡"还是"狗"为例

编码方式

- 错误编码：选择 $y \in \{1, 2, 3\}$ ，分别代表{狗, 猫, 鸡}
 - 原因：无形之间引入了关联，如 狗+猫=鸡
- 正确编码：独热编码
 - 一个向量，它的分量和类别一样多。类别对应的分量设置为1，其他所有分量设置为0。
 - 其中(1, 0, 0)对应于“猫”、(0, 1, 0)对应于“鸡”、(0, 0, 1)对应于“狗”：

$$y \in \{(1, 0, 0), (0, 1, 0), (0, 0, 1)\}.$$

网络架构

- 4个特征与3个输出类别
 - 即输入层4个，输出层3个
- 12个 w ，3个 b ，以及3个未规范化的预测 o_1 、 o_2 和 o_3

$$o_1 = x_1w_{11} + x_2w_{12} + x_3w_{13} + x_4w_{14} + b_1,$$

$$o_2 = x_1w_{21} + x_2w_{22} + x_3w_{23} + x_4w_{24} + b_2,$$

$$o_3 = x_1w_{31} + x_2w_{32} + x_3w_{33} + x_4w_{34} + b_3.$$

 softmax回归是一种单层神经网络

$$\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

- $\mathbf{W}$ 为 3×4 的矩阵

参数开销

- 任何具有 d 个输入和 q 个输出的全连接层，参数开销为 $\mathcal{O}(dq)$
- 将 d 个输入转换为 q 个输出的成本可以减少到 $\mathcal{O}(\frac{dq}{n})$ ，其中超参数 n 可以由我们灵活指定

运算

输出 $\hat{y}_j$ 可以视为属于类 j 的概率， $\operatorname{argmax}_j y_j$ 作为我们的预测，不过得先对预测 $\mathbf{o}$ 进行校准

- 校准目的
 - 使概率总和为1
 - 不为负值
 - 使模型可导

$$\hat{\mathbf{y}} = \operatorname{softmax}(\mathbf{o}) \quad \text{其中} \quad \hat{y}_j = \frac{\exp(o_j)}{\sum_k \exp(o_k)}$$

小批量样本的矢量化

假设我们读取了一个批量的样本 $\mathbf{X}$ ，其中特征维度（输入数量）为 d ，批量大小为 n 。此外，假设我们在输出中有 q 个类别。那么小批量样本的特征为 $\mathbf{X} \in \mathbb{R}^{n \times d}$ ，权重为 $\mathbf{W} \in \mathbb{R}^{d \times q}$ ，偏置为 $\mathbf{b} \in \mathbb{R}^{1 \times q}$ 。softmax回归的矢量计算表达式为：

$$\begin{aligned} \mathbf{O} &= \mathbf{X}\mathbf{W} + \mathbf{b}, \\ \hat{\mathbf{Y}} &= \operatorname{softmax}(\mathbf{O}). \end{aligned}$$

相对于一次处理一个样本，小批量样本的矢量化加快了 $\mathbf{X}$ 和 $\mathbf{W}$ 的矩阵-向量乘法。由于 $\mathbf{X}$ 中的每一行代表一个数据样本，那么softmax运算可以**按行**执行：对于 $\mathbf{O}$ 的每一行，我们先对所有项进行幂运算，然后通过求和对它们进行标准化。 $\mathbf{X}\mathbf{W} + \mathbf{b}$ 的求和会使用广播机制，小批量的未规范化预测 $\mathbf{O}$ 和输出概率 $\hat{\mathbf{Y}}$ 都是形状为 $n \times q$ 的矩阵，每一行的数值代表该行样本的分类概率

对数似然&交叉熵损失

- $\hat{\mathbf{y}}$ 可以视为“对给定任意输入 $\mathbf{x}$ 的每个类的条件概率”。
 - 例如， $\hat{y}_1 = P(y = \text{猫} \mid \mathbf{x})$
- 假设整个数据集 $\{\mathbf{X}, \mathbf{Y}\}$ 具有 n 个样本，其中索引 i 的样本由特征向量 $\mathbf{x}^{(i)}$ 和独热标签向量 $\mathbf{y}^{(i)}$ 组成

$$P(\mathbf{Y} \mid \mathbf{X}) = \prod_{i=1}^n P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}).$$

要最大化 $P(\mathbf{Y} \mid \mathbf{X})$ ，则需要最小化负对数似然

$$-\log P(\mathbf{Y} \mid \mathbf{X}) = \sum_{i=1}^n -\log P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}) = \sum_{i=1}^n l(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}),$$

其中，对于任何标签 $\mathbf{y}$ 和模型预测 $\hat{\mathbf{y}}$ ，损失函数为：

$$l(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^q y_j \log \hat{y}_j.$$

由于 y 是长为 q 的独热编码向量，所以除了目标项以外的 y_j 都是0，例如：

$$\begin{aligned} l(y, \hat{y}) &= - [y_1 * \log(\hat{y}_1) + y_2 * \log(\hat{y}_2) + y_3 * \log(\hat{y}_3)] \\ &\quad \uparrow \qquad \qquad \uparrow \qquad \qquad \uparrow \\ &\quad y_1=1 \qquad y_2=0 \qquad y_3=0 \\ &= - [1 * \log(0.7) + 0 * \log(0.2) + 0 * \log(0.1)] \\ &= - \log(0.7) \\ &= 0.357 \end{aligned}$$

代码

相关名词

- num_worker = 4：4 个进程提前把数据加载到内存，训练进程直接从内存拿
- batch_size = 256：就是每次训练看 256 张图片，算一次梯度，更新一次参数。

区分于线性回归的代码

- 初始化模型

```
net = nn.Sequential(nn.Flatten(),nn.Linear(784,10)) # 即784个输入，10个输出
```

```
def init_weights(m):  
    if type(m) == nn.Linear:  
        nn.init.normal_(m.weight, std=0.01)
```

```
net.apply(init_weights)
```

- 损失函数——交叉熵

```
loss = nn.CrossEntropyLoss(reduction='none')
```